

Practical Linux Administration & NVIDIA GPU Management

Enterprise Systems, Virtualization & High-Performance Computing

2-Day Training

Dual-Distribution: **Ubuntu 24.04 LTS & Rocky Linux 9**

Datacenter Compute: **NVIDIA GPU Administration & Multi-Instance GPU (MIG)**

Daily Schedule: 10:00 – 16:00 | Lunch Break: 13:00 – 14:00

JUL

17

2-Day Training Timetable & Syllabus

Day	Time	Module	Core Topics & Practical Focus
Day 1	10:00 – 11:30	Module 1	Terminal Basics: UNIX philosophy, shell prompt, streams, paths, nano.
	11:30 – 13:00	Module 2	File Structure & Security: FHS tree, user/group models, sudo, octal permissions.
	13:00 – 14:00	LUNCH	<i>Mid-day Break</i>
	14:00 – 15:00	Module 3	Package Management: APT (<code>.deb</code>) vs DNF (<code>.rpm</code>), repo architecture, updates.
	15:00 – 16:00	Module 4	Storage & Filesystems: Block devices, <code>df</code> / <code>du</code> , loopback disks, ext4 vs XFS.
Day 2	10:00 – 11:30	Module 5	Processes & Systemd: Process model, signals, PID 1, systemd unit lifecycle.
	11:30 – 13:00	Module 6	Networking & SSH: TCP/IP sockets, ports, asymmetric cryptography, Ed25519.
	13:00 – 14:00	LUNCH	<i>Mid-day Break</i>
	14:00 – 15:00	Module 7	Security & Logs: Centralized logging, <code>systemd-journald</code> , <code>tail -f</code> , filtering.
	15:00 – 16:00	Module 8	NVIDIA GPU & MIG: GPU architecture, <code>nvidia-smi</code> metrics, hardware MIG slicing.

Lab Architecture: Dual-Distro + Datacenter GPU

Local Virtualization (Canonical Multipass)

- **Ubuntu 24.04 LTS (`ubuntu`)**
 - Standard in modern cloud, AI developer tooling, and containers.
 - Package manager: `apt` / `dpkg` .
 - Default filesystem: `ext4` .
- **Rocky Linux 9 (`rockylinux`)**
 - RHEL 9 binary-compatible enterprise standard.
 - Package manager: `dnf` / `rpm` .
 - Default filesystem: `xfs` .
- Isolated hypervisor bridge network.

Remote AI/HPC Server

- **Bare-Metal NVIDIA GPU Server**
 - Enterprise Datacenter GPU (A100 / H200).
 - Production NVIDIA Linux Kernel Drivers & NVML.
 - Silicon-level Multi-Instance GPU (MIG) capability.
 - Dedicated participant SSH access for Module 8.
- Simulates real-world production AI cluster nodes.

[!NOTE]

Learning both the Debian and RHEL ecosystems equips administrators to handle virtually any corporate Linux server environment.

DAY 1: Foundation & Daily File Management

Module 1: Terminal Basics in Multipass

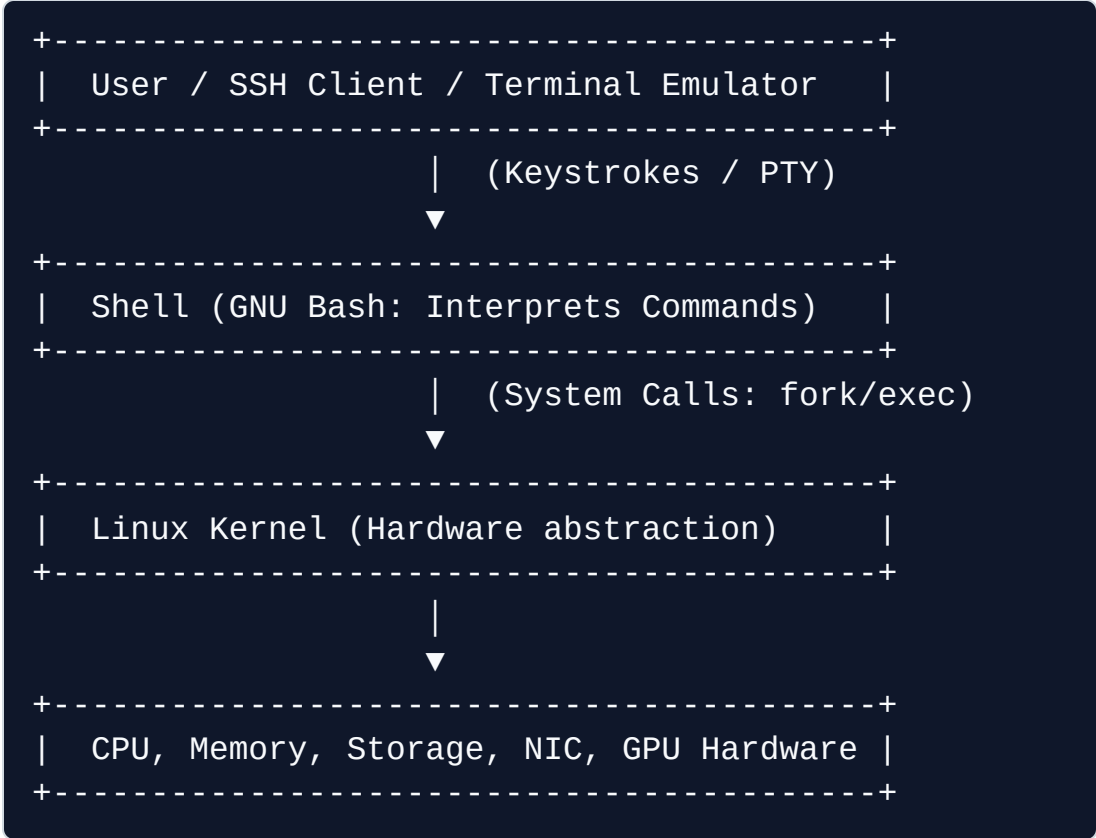
The UNIX Philosophy • Terminal Architecture • Shell Streams • Navigation • Nano

The UNIX Philosophy & Shell Architecture

Core UNIX Principles (McIlroy & Ritchie)

- 1. **Do one thing and do it well:** Modular utilities.
- 2. **Universal text streams:** Output of one tool becomes input to another.
- 3. **Flat text files:** Human-readable, versionable configurations.
- 4. **Everything is a file:** Storage, hardware devices, and network sockets are accessed through the VFS.

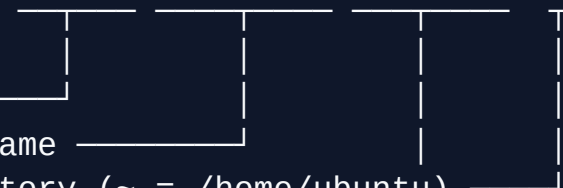
System Interaction Architecture



Anatomy of the Terminal Prompt

In modern Linux distributions, the shell prompt displays four essential operational indicators:

```
ubuntu@server-01:~/projects$
```



1. Current User
2. Machine Hostname
3. Working Directory (~ = /home/ubuntu)
4. Privilege Level (\$ = Standard User, # = Root/Admin)

Standard User (\$)

- Sandboxed execution permissions.
- Restricted to personal home directory (~).
- Cannot alter system configuration or delete core OS files without explicit privilege elevation (`sudo`).

Superuser / Root (#)

- Unrestricted kernel-level capabilities.
- Bypasses standard POSIX permission checks.
- **Enterprise Best Practice:** Avoid permanent root logins. Use `sudo` to maintain accountability and audit logs.

Path Traversal: Absolute vs Relative Paths

All directories in Linux branch from a single inverted tree rooted at `/`.

Absolute Paths

- Always start with the root directory (`/`).
- Point to the exact same file regardless of where you currently are.
- Mandatory for automated scripts, crontabs, and systemd units.
- **Examples:**
 - `/etc/nginx/nginx.conf`
 - `/var/log/syslog`
 - `/home/ubuntu/training/notes.txt`

Relative Paths

- Defined relative to the current working directory (`pwd`).
- Shorter to type during interactive terminal work.
- Uses navigational shortcuts:
 - `.` : Current directory (`./run.sh`)
 - `..` : Immediate parent directory (`cd ..`)
 - `~` : Current user's home directory (`~/notes.txt`)
 - `-` : Previous working directory (`cd -`)

```
# Equivalent ways to reach the same directory:
cd /home/ubuntu/training/day1      # Absolute path
cd ~/training/day1                 # User home relative
cd ../day1                         # Relative from /home/ubuntu/other
```

Command Syntax & The Three Standard Streams

Linux commands follow a standard format: `command [-flags] [arguments]`

Processes communicate with the environment using three standard POSIX file descriptors:

standard input (stdin)

- **FD 0**
- Input data stream.
- Defaults to keyboard.
- Redirect with `<`:

```
mysql db < dump.sql
```

standard output (stdout)

- **FD 1**
- Normal output stream.
- Defaults to screen.
- Redirect with `>` (overwrite) or `>>` (append):

```
ls -la > list.txt
```

standard error (stderr)

- **FD 2**
- Diagnostic & error logs.
- Defaults to screen.
- Redirect with `2>` or merge with stdout using `2>&1`:

```
cmd > log 2>&1
```

The UNIX Pipeline (|)

Connects the **stdout** of one process directly to the **stdin** of another via kernel memory:

```
cat /var/log/syslog | grep "Failed password" | awk '{print $11}' | sort | uniq -c
```



Headless Editing: GNU Nano Essentials

Most enterprise Linux servers operate without a Graphical User Interface (headless). Text configuration files must be inspected and edited directly via the CLI.

GNU nano Characteristics

- Pre-installed across Debian/Ubuntu and RHEL/Rocky.
- Modeless editor (keystrokes directly insert text).
- Shortcut key conventions displayed at the bottom:
 - `^` indicates the **Ctrl** key.
 - `M-` indicates the **Alt / Meta** key.

Core Shortcut Cheat Sheet

- `Ctrl + O` : **WriteOut** (Save changes to disk).
- `Ctrl + X` : **Exit** (Prompts to save if modified).
- `Ctrl + W` : **Where Is** (Search text).
- `Ctrl + K` : **Cut Line** (Delete current line).
- `Ctrl + U` : **Uncut Line** (Paste cut line).
- `Ctrl + _` : **Jump to line number**.

[!TIP]

Hands-On Lab 1 Milestone: Launch the Multipass Ubuntu shell (`multipass shell ubuntu`), practice path navigation, inspect hidden dotfiles with `ls -la`, and create configuration files using `nano`.

DAY 1: Foundation & Daily File Management

Module 2: File Structure, Users & Permissions

Filesystem Hierarchy Standard (FHS) • Users & Groups • Sudo Architecture • Permissions

Filesystem Hierarchy Standard (FHS)

Linux organizes all storage, devices, and filesystems into a single unified directory tree under `/`:

```
/ (Root Directory)
├── bin -> usr/bin      # Fundamental user command binaries (ls, cp, bash)
├── boot                # Linux kernel images (vmlinuz), initramfs, GRUB bootloader
├── dev                 # Device nodes representing hardware (/dev/sda, /dev/null, /dev/nvidia*)
├── etc                 # Host-specific system-wide text configurations (nginx, ssh, passwd)
├── home                # User personal workspaces (/home/ubuntu, /home/alex)
├── lib -> usr/lib       # Shared system libraries (.so) required by binaries
├── media / mnt         # Mount points for removable media and manual filesystem mounts
├── opt                 # Add-on third-party software packages
├── proc                # Virtual pseudo-filesystem exposing live kernel & process state
├── root                # Home directory of the root superuser (NOT /home/root!)
├── sys                 # Virtual pseudo-filesystem exposing device drivers & hardware hierarchy
├── tmp                 # Ephemeral temporary scratch space (wiped on reboot)
├── usr                 # Secondary hierarchy for user utilities, libraries, and docs
└── var                 # Variable operational data: logs (/var/log), spool, caches, web roots
```



Multi-User Architecture & Security Boundaries

Linux isolates users and daemons to protect the operating system from unauthorized changes or security exploits.

1. Account Tiers

- **Root (UID 0):** Superuser with unrestricted access.
- **System Accounts (UID 1–999):** Unprivileged service daemons (`nginx` , `sshd`). No login shell.
- **Regular Users (UID 1000+):** Human administrators and developers.

2. Core Auth Databases

- `/etc/passwd` : Publicly readable user catalog (Username, UID, GID, Home, Shell).
- `/etc/shadow` : Restricted file (mode 600/000) containing salted password hashes.
- `/etc/group` : Defines group names and member lists.

3. User Commands

- `useradd -m -s /bin/bash <user>` : Create user with home directory and shell.
- `passwd <user>` : Set or update password.
- `usermod -aG <group> <user>` : Append user to secondary group.
- `id <user>` : Inspect active UID, GID, and groups.

Privilege Escalation: The `sudo` Subsystem

`sudo` (*Superuser Do*) allows authorized users to execute specific commands with root privileges while logging actions.

Why Avoid Shared Root Logins?

- Shared root passwords prevent auditing—impossible to identify *who* made changes.
- Password revocation requires changing credentials across all hosts.
- `sudo` authenticates using the **user's own password**.
- All administrative actions are recorded in security logs (`/var/log/auth.log` or `/var/log/secure`).

Distro Differences: Sudo Groups

Ubuntu and Rocky Linux use different default administrative groups defined in `/etc/sudoers`:

- **Ubuntu / Debian Family:**

```
sudo usermod -aG sudo alex
```

- **Rocky Linux / RHEL Family:**

```
sudo usermod -aG wheel alex
```

(Derived from the historic Unix term "wheel" group)

[!WARNING]

Always edit `/etc/sudoers` using `sudo visudo`. It performs syntax validation before saving, preventing syntax errors from locking administrators out of the system.

POSIX Permission Architecture: Decoded

Every file and directory in Linux has an owner (**User**), an owning **Group**, and a 9-bit permission mode string:

- r w x r - x r - -

1

ubuntu

staff

4096

Sep 7 10:00

deploy.sh

File Type

User (Owner)

Group Members

Others (World)

Group Owner

User Owner

Permissions on Files

- **Read (r = 4)**: View file contents (`cat` , `nano`).
- **Write (w = 2)**: Modify or overwrite file contents.
- **Execute (x = 1)**: Run the file as a program or script.

Permissions on Directories

- **Read (r = 4)**: List filenames inside the folder (`ls`).
- **Write (w = 2)**: Create, rename, or delete files inside!
- **Execute (x = 1)**: Enter / `cd` into folder or access inodes inside!

Octal Permission Math & Enterprise Modes

Permissions are calculated by summing the octal values of each 3-bit triplet:

$$\text{Octal Value} = \text{Read}(4) + \text{Write}(2) + \text{Execute}(1)$$

Triplet	Binary	Octal	Access Level
<code>rwX</code>	<code>111</code>	7	Full Read, Write & Execute
<code>rw-</code>	<code>110</code>	6	Read & Write (Normal data file)
<code>r-X</code>	<code>101</code>	5	Read & Execute (Programs & dirs)
<code>r--</code>	<code>100</code>	4	Read-Only
<code>---</code>	<code>000</code>	0	No access permissions

```
# Applying permissions and ownership:
chmod 755 script.sh      # rwxr-xr-x
chmod 644 config.yaml    # rw-r--r--
chmod 600 id_ed25519     # rw-----
chown alex:developers project/
```

Common Enterprise Modes

- **Mode 755 (`rwxr-xr-x`):** Executable scripts, public directories, web roots.
- **Mode 644 (`rw-r--r--`):** Configuration files, static assets, documents.
- **Mode 600 (`rw-----`):** Sensitive credentials, private SSH keys, TLS certificates.
- **Mode 700 (`rwx-----`):** Private user home directories or `~/.ssh`.

[!TIP]
Hands-On Lab 2 Milestone: Create user `alex`, configure sudo/wheel privileges, and apply octal permissions to protect sensitive files.

LUNCH BREAK (13:00 – 14:00)

Morning Wrap-up & Afternoon Preview

Mid-day Break

Afternoon Sessions:

- **Module 3 (14:00 – 15:00):** Package Management (`apt` vs `dnf` , Repositories, Updates)
- **Module 4 (15:00 – 16:00):** Storage & Virtual Filesystem Labs (Block Devices, Loopback, ext4 vs XFS)

DAY 1: Foundation & Daily File Management

Module 3: Package Management (APT vs DNF)

Package Architecture • Repositories • Metadata Indexing • APT vs DNF Rosetta Stone

Why Package Managers? Overcoming Dependency Hell

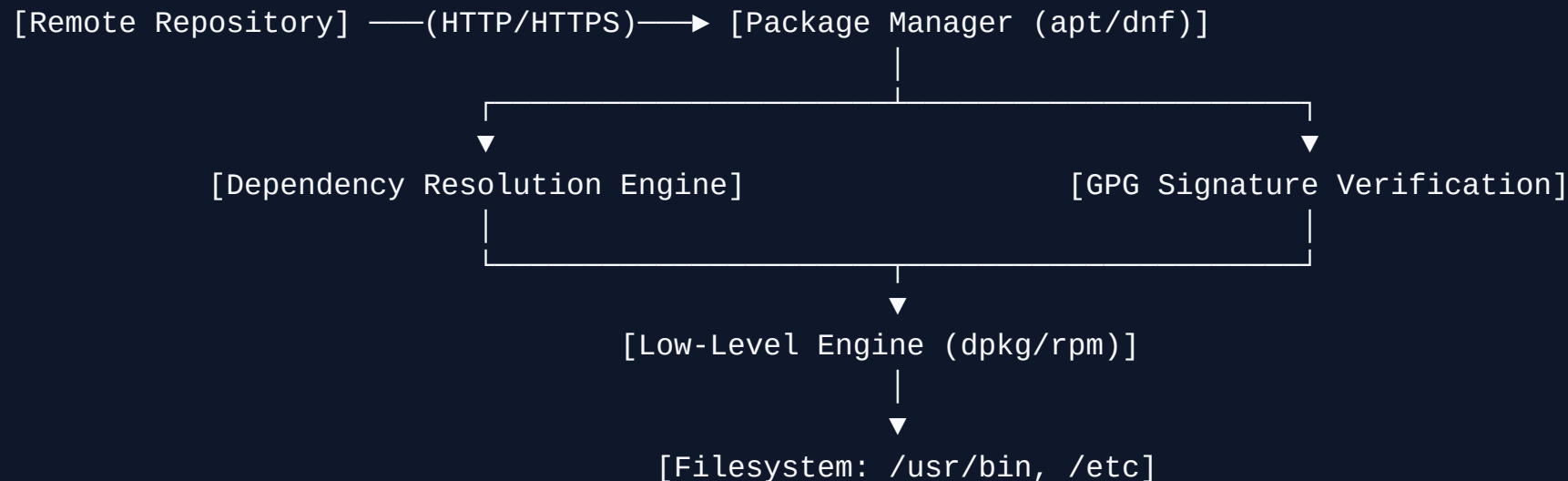
Modern package managers automate downloading, verifying, configuring, and updating software.

✗ Manual Source Compilation

- Compiling from source required manual dependency resolution (`make` , `gcc`).
- Updating libraries frequently broke existing software.
- No central catalog for cleanly uninstalling or verifying software integrity.

✓ Modern Package Management

- Pre-compiled binaries targeted to CPU architectures (`x86_64` , `aarch64`).
- Automatic dependency graph calculation.
- Cryptographic verification via GPG repository signatures.
- Clean software lifecycle: install, upgrade, rollback, remove.



Debian/Ubuntu vs RHEL/Rocky Architecture

● Ubuntu / Debian (apt + dpkg)

- **Package Format:** `.deb`
- **Low-Level Tool:** `dpkg` (installs local `.deb` files; does not fetch remote dependencies).
- **High-Level Tool:** `apt` / `apt-get` (fetches from mirrors, calculates dependency tree).
- **Configuration:** `/etc/apt/sources.list` and `/etc/apt/sources.list.d/`
- **Package Cache:** `/var/cache/apt/archives/`

● Rocky Linux / RHEL (dnf + rpm)

- **Package Format:** `.rpm` (Red Hat Package Manager)
- **Low-Level Tool:** `rpm` (unpacks payloads, manages RPM database).
- **High-Level Tool:** `dnf` (*Dandified YUM*, SAT-based dependency solver).
- **Configuration:** `/etc/yum.repos.d/*.repo`
- **Package Cache:** `/var/cache/dnf/`

[!IMPORTANT]

Key Concept: `apt update` does **not** upgrade installed packages! It only downloads the latest index metadata from upstream mirrors. You must follow with `apt upgrade` to apply updates.

Package Management: Side-by-Side Rosetta Stone

Operational Task	Ubuntu 24.04 (apt)	Rocky Linux 9 (dnf)
Refresh repository metadata	<code>sudo apt update</code>	<code>sudo dnf check-update</code>
Upgrade installed packages	<code>sudo apt upgrade -y</code>	<code>sudo dnf upgrade -y</code>
Search for available package	<code>apt search <package></code>	<code>dnf search <package></code>
Display package details	<code>apt show <package></code>	<code>dnf info <package></code>
Install package(s)	<code>sudo apt install -y <pkg></code>	<code>sudo dnf install -y <pkg></code>
Remove package (keep configs)	<code>sudo apt remove <pkg></code>	<code>sudo dnf remove -y <pkg></code>
Purge package + configs	<code>sudo apt purge <pkg></code>	<code>sudo dnf remove -y <pkg></code>
Remove unused dependencies	<code>sudo apt autoremove -y</code>	<code>sudo dnf autoremove -y</code>
Find which package owns file	<code>dpkg -S /path/to/file</code>	<code>rpm -qf /path/to/file</code>
Clean local package cache	<code>sudo apt clean</code>	<code>sudo dnf clean all</code>

[!TIP]
Hands-On Lab 3 Milestone: Install and verify `nginx` and `htop` simultaneously on Ubuntu (`apt`) and Rocky Linux (`dnf`).

DAY 1: Foundation & Daily File Management

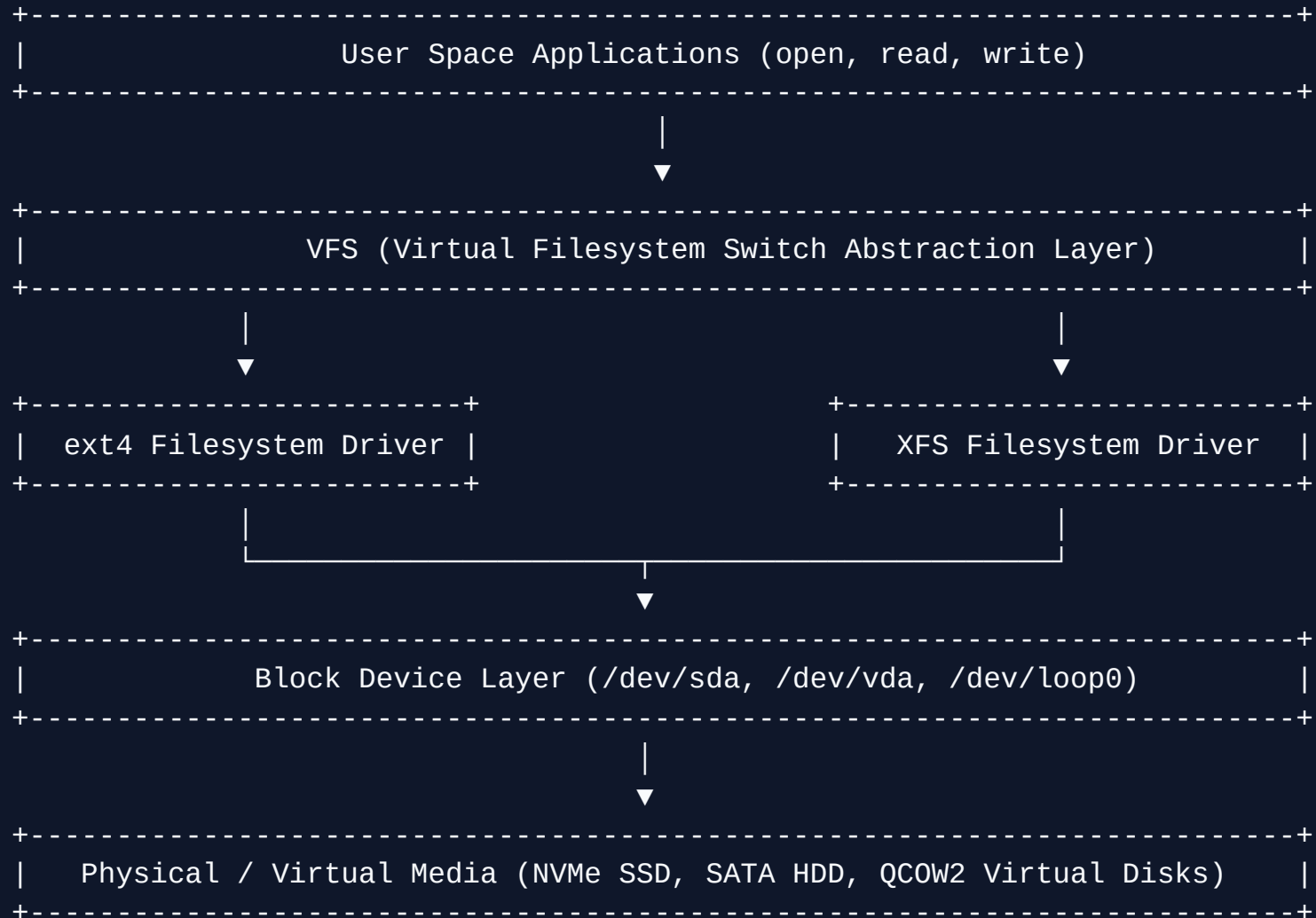
Module 4: Storage & Virtual Filesystem Labs

Storage Stack • Telemetry (df vs du) • Loopback Disks • ext4 vs XFS • Mount Lifecycle



The Linux Storage Architecture Stack

Linux uses the **Virtual Filesystem Switch (VFS)** to present heterogeneous storage hardware through a uniform file interface:



Storage Telemetry: `df` vs `du`

Understanding the difference between filesystem-level and file-level metrics is critical for disk troubleshooting:

`df -h` (Disk Free)

- Queries filesystem metadata from the **superblock**.
- Instantaneous output regardless of filesystem size.
- Reports capacity in human-readable units (`-h`).
- Displays mount points, total blocks, used blocks, and available inodes (`df -i`).

`du -sh` (Disk Usage)

- Recursively traverses directories file by file.
- Calculates actual disk space consumed by files.
- Slower on large directory trees.
- **Top space consumer command:**

```
sudo du -sh /var/* | sort -h
```

[!CAUTION]

Deleted Open File Phenomenon: If an application holds a file open when an admin deletes it with `rm`, the directory entry is removed, but disk blocks stay allocated until the process terminates! `df` reports full disk while `du` shows empty. Use `sudo lsof +L1` to identify unlinked open files.

Loopback Block Devices & Filesystems: ext4 vs XFS

A **loopback device** (`/dev/loop*`) allows mounting a regular file on disk as if it were a physical block device:

Virtual Disk Creation Pipeline

- 1. **Allocate disk image with `dd` :**
`dd if=/dev/zero of=disk.img bs=1M count=100`
- 2. **Format filesystem structure:**
 - Ubuntu standard: `mkfs.ext4 disk.img`
 - Rocky Linux standard: `mkfs.xfs disk.img`
- 3. **Mount to VFS directory:**
`mount -o loop disk.img /mnt/virtual_storage`

Filesystem Comparison

Feature	ext4 (Fourth Extended)	XFS (High-Performance)
Default OS	Ubuntu / Debian	Rocky / RHEL
Primary Focus	General-purpose stability	High-throughput concurrency
Max File Size	16 TiB	8 EiB
Shrink Support	Yes (<code>resize2fs</code>)	No (Cannot shrink XFS)
Metadata	Block groups	Allocation groups

Mounting & The Filesystem Table (/etc/fstab)

Mounting attaches a formatted storage device to a designated directory (mount point):

Mount Operations

```
# Create mount directory:
sudo mkdir -p /mnt/virtual_storage

# Mount loopback image:
sudo mount -o loop /var/virtual_disk.img /mnt/virtual_storage

# Write data into the mounted disk:
echo "UKM Training" | sudo tee /mnt/virtual_storage/test.txt

# Inspect active mounts:
mount | grep virtual_storage

# Clean unmount (flushes cached buffers to disk):
sudo umount /mnt/virtual_storage
```

/etc/fstab (Static Mount Table)

Manual `mount` commands do not survive reboot.

Permanent mounts must be declared in `/etc/fstab` with 6 fields:

# <device>	<mount point>	<type>	<options>	<dump>	<pass>
UUID=3a8f...-01	/data	xfs	defaults	0	2

1. Device Identifier (UUID or `/dev/sdX`)
2. Target Mount Point
3. Filesystem Type (`ext4` , `xfs` , `nfs`)
4. Mount Options (`defaults` , `ro` , `noexec`)
5. Dump backup utility flag (`0` to disable)
6. Filesystem check order (`1` for root, `2` for others)

[!TIP]

Hands-On Lab 4 Milestone: Create a 100MB loopback file, format it with `ext4` / `xfs` , and mount it to `/mnt/virtual_storage` .

🏁 Day 1 Wrap-up & Core Competencies Mastered

✅ Day 1 Milestones Completed

1. Headless terminal navigation, standard streams, and pipelines.
2. File operations and safe CLI editing with `nano`.
3. Navigating the Linux FHS directory layout.
4. User management, sudo security, and octal permissions (`chmod 755/644/600`).
5. Dual-distro package management (`apt` vs `dnf`).
6. Disk telemetry with `df` / `du`, loopback virtual devices, and mounting.

🚀 Day 2 Roadmap

- **Module 5:** Process management, kill signals, and systemd service control.
- **Module 6:** Network diagnostics, port binding, and Ed25519 SSH keys.
- **Module 7:** System observability with `systemd-journald`, `tail -f`, and `grep`.
- **Module 8:** Enterprise NVIDIA GPU server telemetry and Multi-Instance GPU (MIG) slicing!

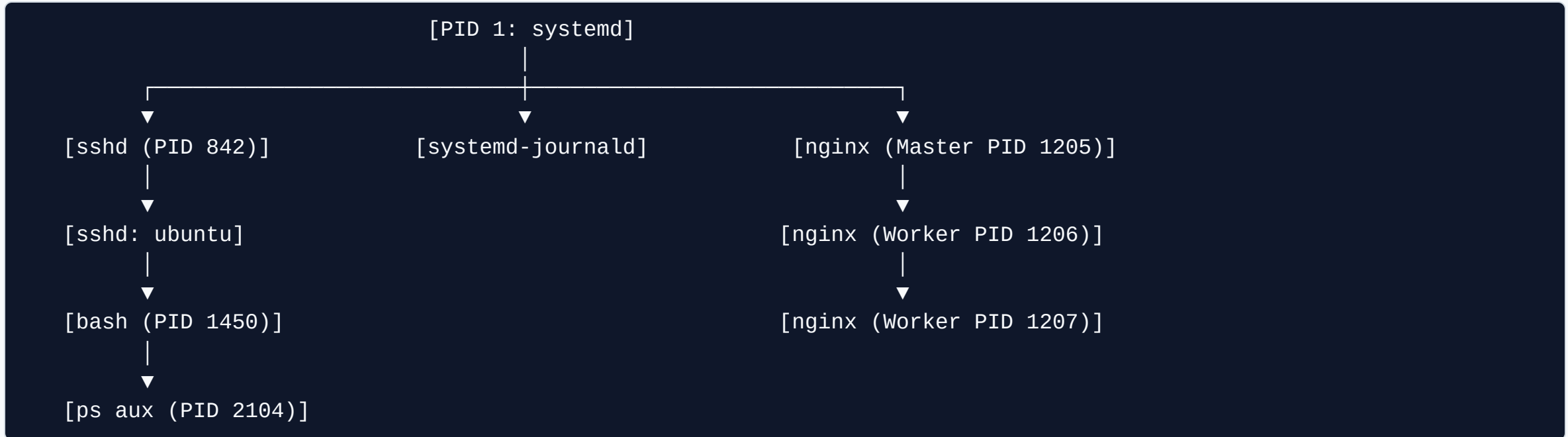
DAY 2: Services, Networking, Security & GPU Admin

Module 5: Processes & Systemd Services

Process Model • Lifecycle States • Kill Signals • Systemd Architecture • Unit Management

The Linux Process Model & Process Tree

A **process** is an actively executing program in memory, with its own address space, open file descriptors, credentials, and CPU execution context:



- **PID (Process ID):** Unique identifier assigned by the Linux kernel.
- **PPID (Parent Process ID):** The process that invoked the child via `fork()`.
- **PID 1 (`systemd`):** The root ancestor of all user-space processes on modern Linux.

Process Lifecycle & Execution States

Processes cycle through five main execution states tracked by the kernel scheduler:

Process States (Reported by `ps` / `top`)

- **R (Running / Runnable):** Executing on CPU or waiting in the active run queue.
- **S (Interruptible Sleep):** Waiting for an event or I/O. Responds to signals.
- **D (Uninterruptible Sleep):** Blocked waiting directly on hardware I/O. Cannot be killed, even with `kill -9`.
- **T (Stopped / Traced):** Suspended by user (`ctrl + Z`) or an attached debugger.
- **Z (Zombie / Defunct):** Terminated, but parent process has not yet read exit status via `waitpid()`. Consumes no RAM, only a PID table slot.

Monitoring Commands

- `ps aux` : Comprehensive snapshot of running processes.
- `ps -ef` : Traditional UNIX format showing PPID relationships.
- `top` / `htop` : Real-time interactive process dashboard.
- `pgrep <name>` : Search active PIDs by process name.
- `pstree -p` : Visual process hierarchy tree showing PIDs.

Inter-Process Signals: Terminating & Reloading

Signals are asynchronous messages sent to a process to notify it of system events or request state changes:

Signal	Name	Behavior & Enterprise Usage
1	SIGHUP	<i>Hangup.</i> Modern daemons reload configuration without dropping active client connections.
2	SIGINT	<i>Interrupt.</i> Generated by typing <code>ctrl + c</code> . Requests polite termination.
9	SIGKILL	<i>Kill.</i> Handled directly by kernel. Cannot be caught or ignored. Immediate teardown.
15	SIGTERM	<i>Terminate.</i> Default signal sent by <code>kill <PID></code> . Allows graceful cleanup and buffer flushing.
18 / 19	CONT / STOP	Resume execution / Suspend process.

Best Practice: Escalation Pattern

Always attempt graceful termination first:

```
# 1. Polite termination request:
sudo kill -15 <PID>

# 2. Wait 5-10 seconds for graceful cleanup...

# 3. Force termination only if hung:
sudo kill -9 <PID>

# Terminate by process name:
sudo killall -15 nginx
sudo pkill -f "python worker.py"
```

Init Evolution: SysVinit to `systemd`

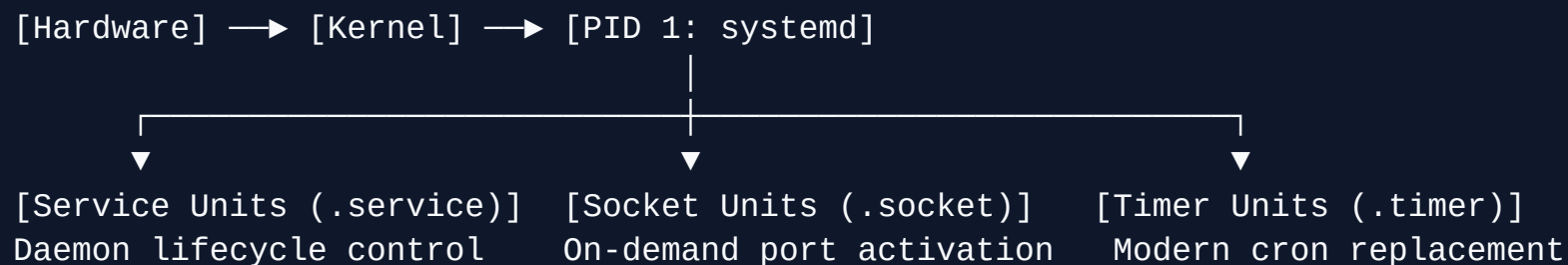
Modern enterprise Linux distributions have adopted `systemd` as the unified initialization system and service manager:

Legacy SysVinit

- Sequential boot: Services started one by one via shell scripts in `/etc/init.d/`.
- Slow boot performance.
- No built-in supervision: Crashed daemons remained dead.
- Child processes could escape tracking by double-forking.

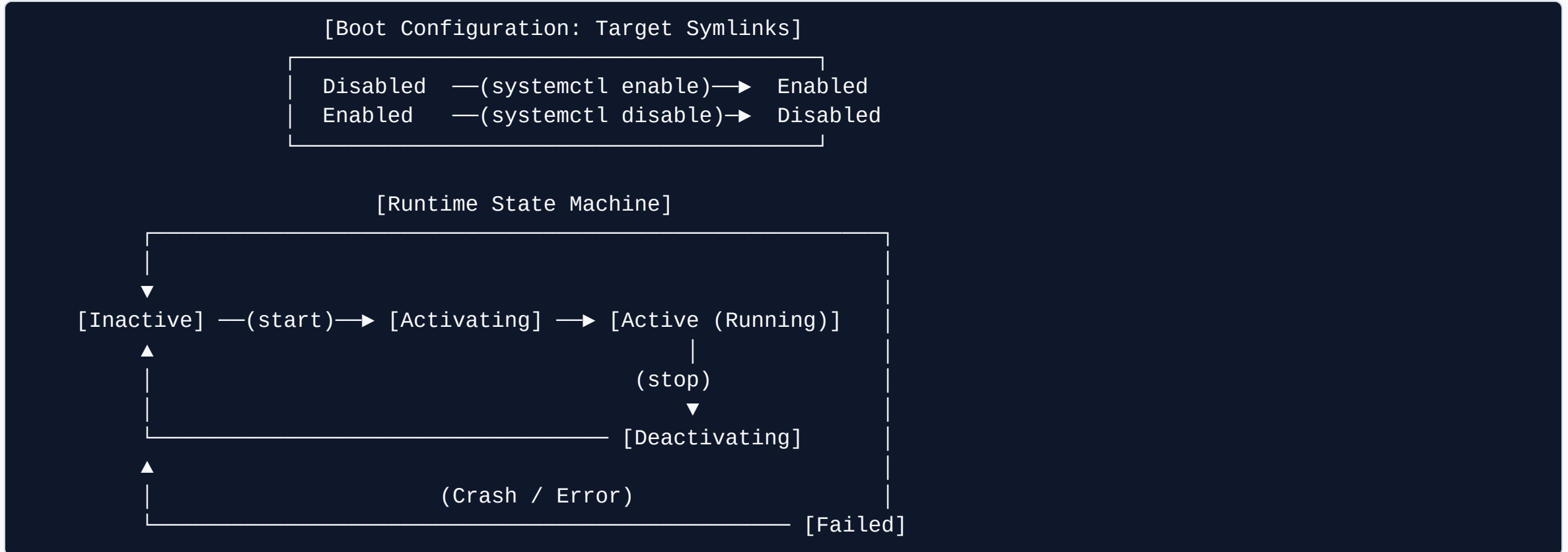
Modern `systemd`

- Concurrent, parallelized boot activation.
- Uses Linux **Control Groups (cgroups)** to track every child process cleanly.
- Automatic restart policies (`Restart=on-failure`).
- Unified operational syntax across Ubuntu, Rocky, Debian, RHEL, and SUSE.



⚙️ Systemd Service Lifecycle & State Machine

Systemd decouples **Runtime Execution** from **Boot Configuration**:



Systemd Management Commands (systemctl)

Operational Task	Command Syntax	Description
Inspect detailed status	<code>systemctl status nginx</code>	Active state, PID, memory usage, cgroup, and recent log lines
Start service immediately	<code>sudo systemctl start nginx</code>	Transitions service from Inactive to Active
Stop running service	<code>sudo systemctl stop nginx</code>	Sends SIGTERM, followed by SIGKILL if timeout expires
Restart service	<code>sudo systemctl restart nginx</code>	Full teardown and re-execution
Reload configuration	<code>sudo systemctl reload nginx</code>	Sends SIGHUP to reload configs without connection drops
Enable on system boot	<code>sudo systemctl enable nginx</code>	Creates target symlink for boot activation
Disable on system boot	<code>sudo systemctl disable nginx</code>	Removes target boot symlink
Enable AND start immediately	<code>sudo systemctl enable --now nginx</code>	Combines <code>enable</code> and <code>start</code>
Check boot enablement	<code>systemctl is-enabled nginx</code>	Returns <code>enabled</code> or <code>disabled</code>
Check runtime state	<code>systemctl is-active nginx</code>	Returns <code>active</code> or <code>inactive</code>

[!TIP]
Hands-On Lab 5 Milestone: Start Nginx, test port responses via `curl`, inspect service status, and verify autostart enablement.

DAY 2: Services, Networking, Security & GPU Admin

Module 6: Networking & SSH in Multipass

TCP/IP Sockets • Port Binding • Asymmetric Cryptography • Ed25519 • Cross-VM SSH

Linux Networking Essentials & Port Binding

Linux provides a kernel-managed network stack configured through user-space tools:

Network Interfaces

- **Loopback (`lo` / `127.0.0.1`)**: Internal interface for local inter-process communication.
- **Physical & Virtual NICs:**
 - `eth0` or persistent names like `enp0s3` .
 - In Multipass: Assigned private IP addresses on an isolated bridge subnet (`10.x.x.x`).

Network Ports & Sockets

A **socket** is an IP address + Port number + Protocol binding (e.g. `0.0.0.0:80/TCP`).

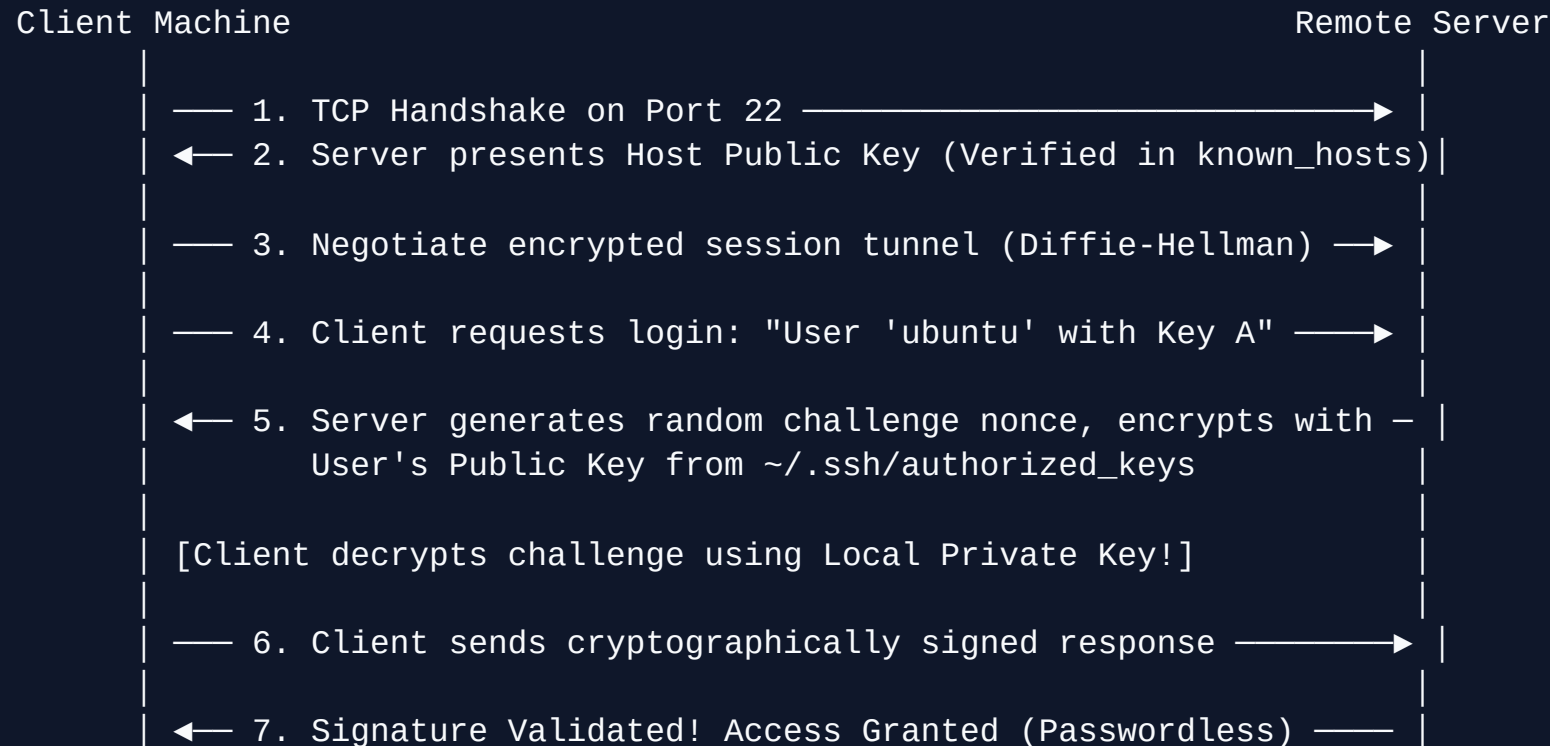
- **Standard Ports:**
 - `22` : SSH (Secure Shell)
 - `80` : HTTP Web Traffic
 - `443` : HTTPS Encrypted Traffic
- **Privileged Ports (0–1023):** Require `root/CAP_NET_BIND_SERVICE` capabilities to bind.

Network diagnostic commands:

```
ip addr show          # View assigned IP addresses (replaces deprecated ifconfig)
ip route show         # View routing table and default gateway
ss -tulpn             # Inspect active listening sockets and owning PIDs
```


SSH Authentication Handshake: Under the Hood

How does a server verify client identity without ever seeing the private key?



Key Algorithms: Why Ed25519 Over RSA?

Modern enterprise infrastructure specifies **Ed25519** (Edwards-curve Digital Signature Algorithm) for SSH authentication:

Ed25519 (Recommended Standard)

- Uses Curve25519 elliptic curve mathematics.
- Compact 256-bit key length provides security equivalent to a 3072-bit RSA key.
- Resistant to side-channel and timing attacks.
- Shorter key string: easier to manage in configuration files and CI/CD pipelines.
- Fast key generation and verification:

```
ssh-keygen -t ed25519 -N "" -f ~/.ssh/id_ed25519
```

RSA (Legacy Standard)

- Relies on integer factorization of large prime numbers.
- Requires at least 3072 or 4096 bits today.
- Larger key payloads; slower computation.
- Older SHA-1 based RSA signatures are disabled by default in modern OpenSSH releases.

[!CAUTION]

Strict Permission Requirements: OpenSSH rejects connection attempts if permissions are too open. Ensure `chmod 700 ~/.ssh` and `chmod 600 ~/.ssh/id_ed25519`.

[!TIP]

Hands-On Lab 6 Milestone: Generate an Ed25519 keypair, configure passwordless login to `localhost`, and SSH between the Ubuntu and Rocky Linux VMs.

LUNCH BREAK (13:00 – 14:00)

Mid-Day Break & Afternoon Preview

Take an hour to recharge and refresh.

Afternoon Sessions:

- **Module 7 (14:00 – 15:00):** Security Essentials & System Logs (`journalctl` , `tail -f` , `grep`)
- **Module 8 (15:00 – 16:00):** NVIDIA GPU Management & Multi-Instance GPU (MIG) on Remote GPU Server!

DAY 2: Services, Networking, Security & GPU Admin

Module 7: Security Essentials & System Logs

Logging Architecture • Syslog vs Journald • Binary Journals • Querying with journalctl • grep

Linux Logging Architecture: Syslog vs Journald

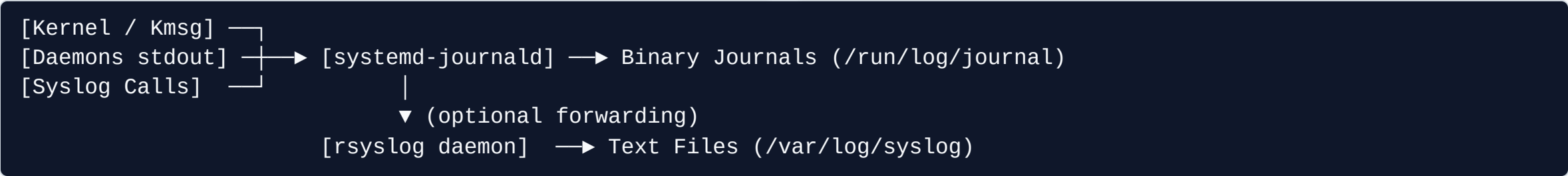
System logs provide essential observability for service debugging, auditing, and incident response:

Traditional Syslog (rsyslog)

- Appends plaintext lines to files in `/var/log/`.
- Standard log destinations:
 - Ubuntu: `/var/log/syslog` , `/var/log/auth.log`
 - Rocky Linux: `/var/log/messages` , `/var/log/secure`
- Managed by `logrotate` to prevent disk filling.
- Queried with standard text tools: `cat` , `grep` , `tail` .

`systemd-journald` (Modern Unified Log)

- Centralized binary ring-buffer logging daemon.
- Aggregates kernel messages, boot events, syslog calls, and service `stdout` / `stderr` .
- Enriches every record with structured metadata (PID, UID, Systemd Unit).
- Fast indexed searches and tamper resistance.
- Queried using the `journalctl` utility.





Structured Log Queries with `journalctl`

Structured journal metadata allows targeted queries without requiring complex pipeline filtering:

Common Query Filters

```
# 1. Filter by specific systemd service:
sudo journalctl -u nginx

# 2. Live stream new log entries:
sudo journalctl -u nginx -f

# 3. View logs from current system boot:
sudo journalctl -b

# 4. View logs from previous boot:
sudo journalctl -b -1

# 5. Filter by time window:
sudo journalctl --since "1 hour ago"
sudo journalctl --since "2026-09-07 10:00:00"
```

Severity Priority Filtering (`-p`)

Journal events follow standard RFC 5424 severity codes:

- `0` : emerg (System unusable)
- `1` : alert (Immediate action required)
- `2` : crit (Critical system conditions)
- `3` : **err (Error conditions)**
- `4` : warning (Warning conditions)
- `5` : notice (Normal but significant)
- `6` : info (Informational)
- `7` : debug (Debug messages)

```
# Display errors and critical faults from current boot:
sudo journalctl -p err -b
```

Real-Time Monitoring & Pipeline Filtering

Sysadmins combine live log following with pattern matching for active incident troubleshooting:

Live File Following (`tail -f`)

Monitors growing log files and streams newly appended lines to the screen:

```
# Follow Nginx HTTP access requests:
sudo tail -f /var/log/nginx/access.log

# Follow Nginx application error logs:
sudo tail -f /var/log/nginx/error.log
```

Press `ctrl + c` to stop following.

Pattern Filtering with `grep`

Searches text streams for matching regular expressions:

- `grep -i "error"` : Case-insensitive matching.
- `grep -n "fail"` : Output line numbers.
- `grep -v "info"` : Invert match (exclude lines).
- `grep -E "404|500"` : Extended regex matching.

```
# Filter for failed SSH login attempts:
sudo grep "Failed password" /var/log/auth.log
```

[!TIP]

Hands-On Lab 7 Milestone: Generate simulated HTTP traffic to Nginx, inspect live logs with `journalctl -f` and `tail -f`, and filter security events using `grep`.

DAY 2: Services, Networking, Security & GPU Admin

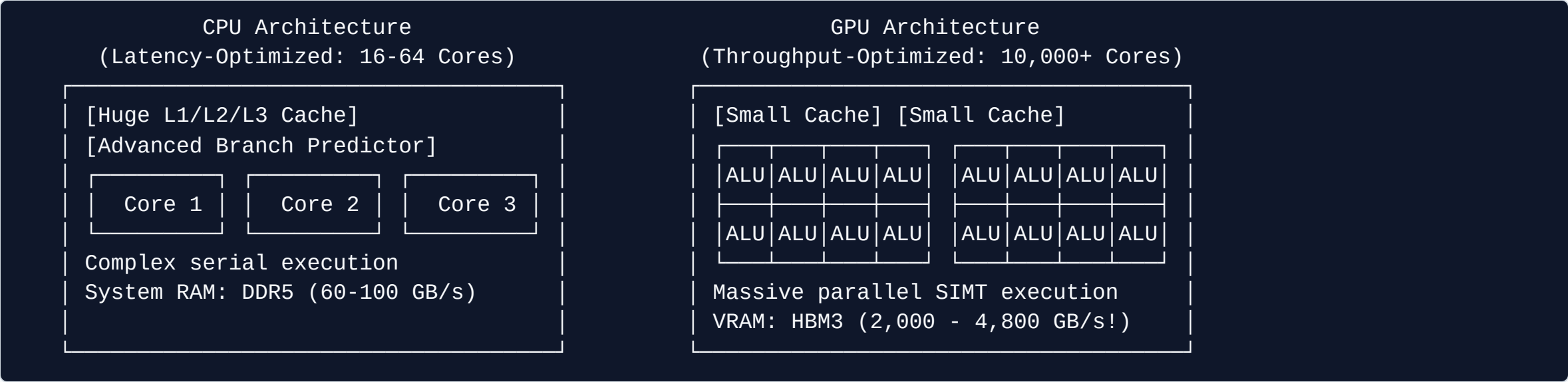
Module 8: NVIDIA GPU Management & MIG

GPU Architecture • Software Stack • nvidia-smi Telemetry • Multi-Instance GPU (MIG)

Environment: Dedicated Remote Server with Enterprise NVIDIA GPUs (A100 / H200)

CPU vs GPU Compute Paradigms in Modern AI

Modern AI model training and LLM inference require massive parallel computation:

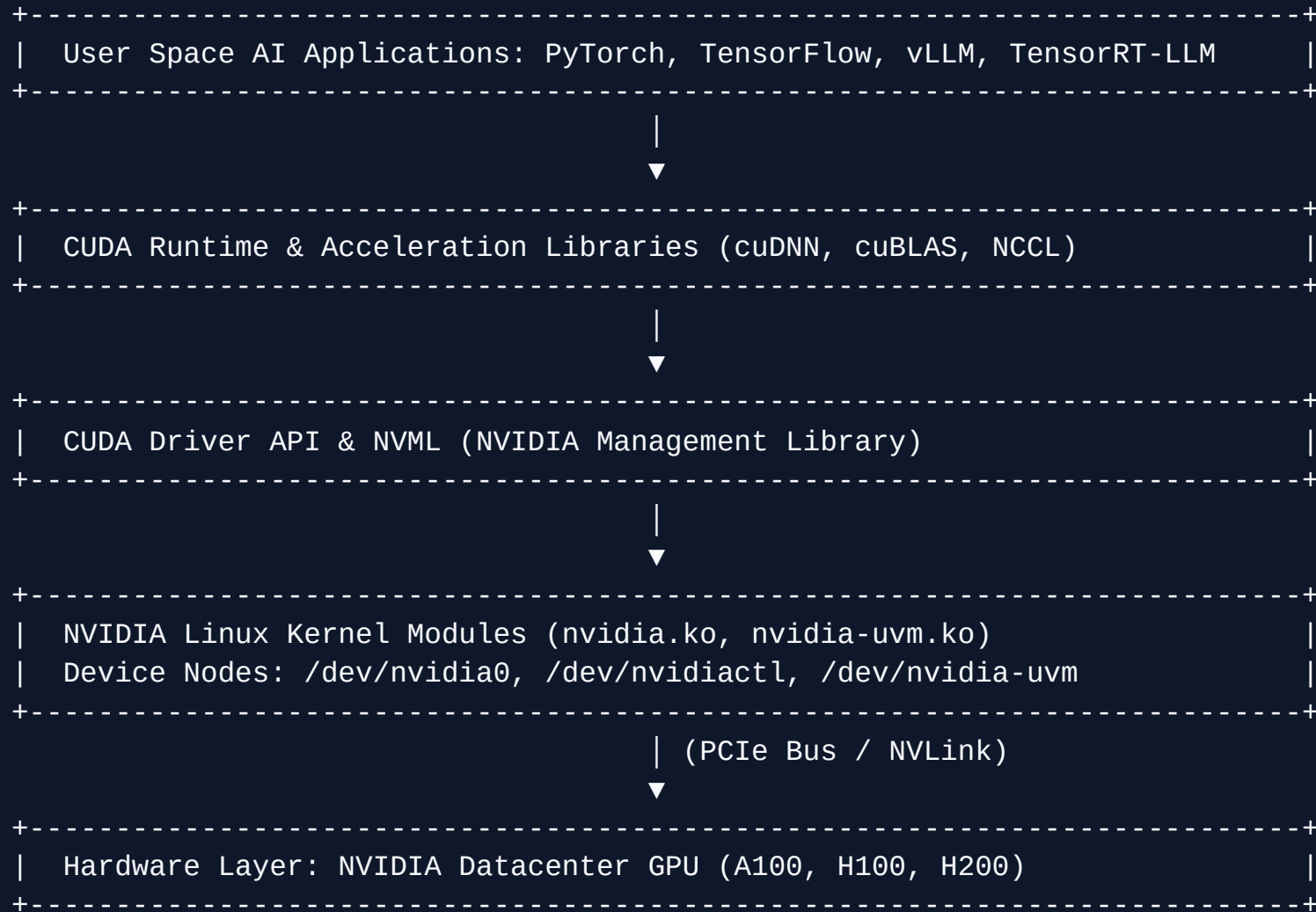


- **CPU:** Optimized for sequential execution, complex control logic, and system orchestration.
- **GPU (SIMT):** Optimized for high-throughput parallel matrix multiplication and tensor arithmetic.



The NVIDIA Linux Software Stack

Accessing NVIDIA GPU hardware from Linux involves several coordinated software layers:





Real-Time GPU Telemetry: `nvidia-smi` Essentials

The **NVIDIA System Management Interface** (`nvidia-smi`) monitors GPU operational status:

```
+-----+
| NVIDIA-SMI 550.54.15                Driver Version: 550.54.15          CUDA Version: 12.4          |
+-----+-----+-----+-----+-----+-----+
| GPU  Name                Persistence-M | Bus-Id                Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |      Memory-Usage      | GPU-Util  Compute M. |
|                                           | MIG M.               |
+=====+=====+=====+=====+=====+=====+
|   0   NVIDIA A100-SXM4-80GB           On   | 00000000:00:04.0 Off  |             0        |
| N/A   38C    P0              68W / 400W | 1420MiB / 81920MiB    |      5%      Default |
|                                           |                       | Enabled         |
+-----+-----+-----+-----+-----+-----+
| Processes:                                |
| GPU   GI    CI          PID    Type    Process name                        GPU Memory |
|       ID    ID                                   |            Usage   |
+=====+=====+=====+=====+=====+=====+
|     0   --   --        14298     C   python3 /workspace/train.py          1410MiB |
+-----+-----+-----+-----+-----+-----+

```

- **Persistence Mode:** Ensures driver stays loaded in kernel memory (`Persistence-M: On`).
- **Power Usage vs Capacity:** Tracks current draw against TDP limit (e.g. `68W / 400W`).
- **Process Types:** `c` = Compute / CUDA application, `G` = Graphics application.



Telemetry Automation & Process Tracking

`nvidia-smi` supports scriptable querying for monitoring pipelines, metrics exporters, and alerting:



Live Terminal Monitor

Continuously refresh GPU status every second:

```
nvidia-smi -l 1
```



Scriptable CSV Telemetry Query

Extract structured metrics for monitoring agents:

```
nvidia-smi --query-gpu=index,name,temperature.gpu,utilization.gpu,memory.used,memory.total --format=csv
```

Example CSV output:

```
index, name, temperature.gpu, utilization.gpu [%], memory.used [MiB], memory.total [MiB]
0, NVIDIA A100-SXM4-80GB, 38, 5 %, 1420 MiB, 81920 MiB
```



Process Tracking

Identify user PIDs consuming GPU VRAM:

```
nvidia-smi --query-compute-apps=gpu_uuid,pid,process_name,used_gpu_memory --format=csv
```



Critical Operational Checks

- **Thermal Limits:** Check for temperatures exceeding 80°C.
- **Out of Memory (OOM):** Track if allocated memory nears capacity.
- **Orphan Processes:** Locate abandoned Python processes holding VRAM after jobs terminate.

The Multi-Tenant Problem in AI Clusters

Enterprise GPUs (e.g. A100-80GB, H200-141GB) represent significant infrastructure investments:

Software Time-Slicing Limitations

- Multiple jobs share a single GPU via software scheduling.
- **The Noisy Neighbor Issue:**
 - One application with an unoptimized memory leak causes a CUDA OOM crash across all concurrent jobs!
 - Unpredictable execution latency.
 - Zero memory hardware isolation.

Enterprise Multi-Tenant Requirements

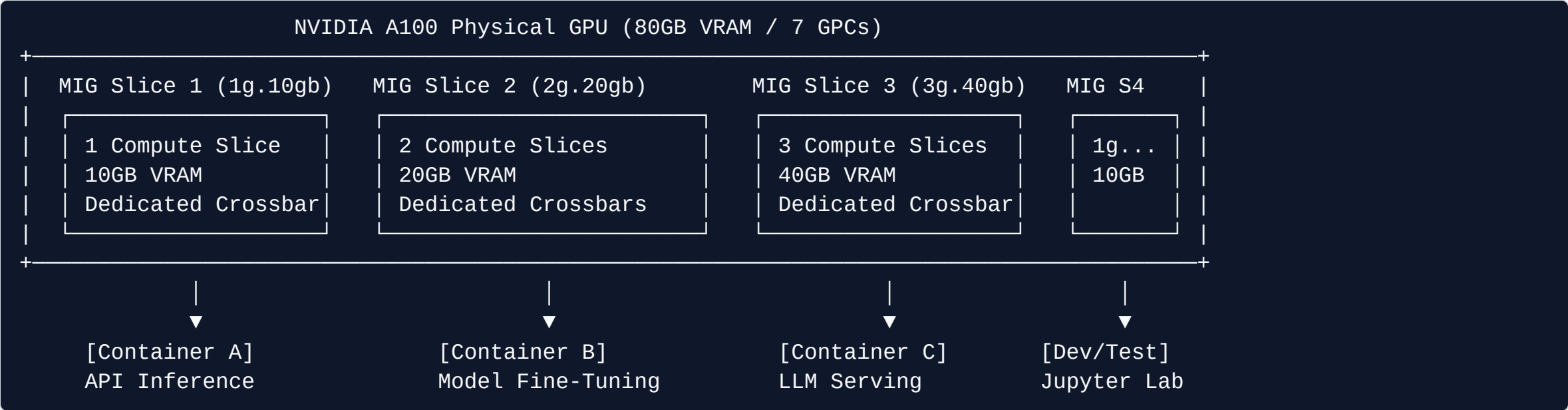
1. **Guaranteed Quality of Service (QoS):** Deterministic latency for real-time inference.
2. **Hard Fault Isolation:** A crash in one workload never affects other workloads.
3. **Hardware Memory Partitioning:** Dedicated, non-overlapping VRAM segments.
4. **Dedicated Compute Paths:** Isolated streaming multiprocessors and cache lines.

[!IMPORTANT]

The Enterprise Solution: NVIDIA **Multi-Instance GPU (MIG)** silicon partitioning technology.

Multi-Instance GPU (MIG): Silicon-Level Partitioning

Available on NVIDIA Ampere (A100) and Hopper (H100/H200) architectures, MIG partitions a single physical GPU into up to **7 hardware-isolated instances**:



- Each instance has dedicated **Streaming Multiprocessors (SMs)**, **Memory Controllers**, and **L2 Cache**.
- Hardware fault isolation guarantees that a crash in one slice does not impact any other slice!

MIG Architecture: GI vs CI & Profile Sizing

MIG defines two hierarchical layers of partitioning:

1. GPU Instance (GI)

- Primary hardware partition.
- Allocates memory controllers, VRAM capacity, and compute slices.

2. Compute Instance (CI)

- Execution slice within a GPU Instance.
- The `-c` flag creates matching 1:1 Compute Instances automatically.



Profile Naming Scheme

{Compute Slices}g. {Memory in GB}gb

- `1g.10gb` : 1 compute slice, 10GB VRAM (Up to 7 instances)
 - Ideal for: Web inference APIs, embeddings, dev testing.
- `2g.20gb` : 2 compute slices, 20GB VRAM (Up to 3 instances)
 - Ideal for: Computer vision, small LLM inference.
- `3g.40gb` : 3 compute slices, 40GB VRAM (Up to 2 instances)
 - Ideal for: Medium model fine-tuning (LoRA), data pipelines.
- `7g.80gb` : Full GPU resources under MIG isolation.



The 4-Phase MIG Operational Lifecycle

Phase 1: Enable MIG Mode → Phase 2: Create Partitions → Phase 3: Bind Workload → Phase 4: Teardown
 (sudo nvidia-smi -mig 1) (nvidia-smi mig -cgi ...) (CUDA_VISIBLE_DEVICES) (nvidia-smi mig -dgi)

Phase 1: Mode Enablement

Requires root privileges:

```
sudo nvidia-smi -i 0 -mig 1
```

Phase 2: Profile Selection & Slicing

List supported profile IDs, then create partitions:

```
# List supported profiles:
nvidia-smi mig -lgip -i 0

# Create a 20GB slice and a 10GB slice:
sudo nvidia-smi mig -cgi 14,19 -C
```

Phase 3: Workload Binding

Find created instance UUIDs:

```
nvidia-smi mig -lgi
```

Bind target application to that slice only:

```
export CUDA_VISIBLE_DEVICES="MIG-<UUID>"
python3 inference_server.py
```

Phase 4: Teardown & Reset

```
# Destroy created MIG instances:
sudo nvidia-smi mig -dgi -i 0

# Disable MIG mode:
sudo nvidia-smi -i 0 -mig 0
```

The 4-Phase MIG Operational Lifecycle (cont.)

[!TIP]

Hands-On Lab 8 Milestone: SSH into the remote dedicated GPU node, query telemetry with `nvidia-smi`, enable MIG mode, carve GPU instances, bind a workload, and perform clean teardown.

Using NVIDIA MIG Parted to create MIG partitions

What is NVIDIA MIG Partition Editor (MIG Parted)

MIG (short for **Multi-Instance GPU**) is a software that allows NVIDIA GPU to be sliced into mini GPUs with a fixed partition of memory and a fixed partition of compute resources.

Why MIG Parted?

1. Less command to run compared to using just nvidia-smi.
2. The configuration can be saved in git repository for backup and sharing purposes.
3. Can be easily paired with systemd to reapply the profile after each reboot.

How to use MIG Parted to create MIG partitions?

1. [Download](#) and install nvidia-mig-parted.
2. List supported MIG profiles for the GPU card

```
sudo nvidia-smi mig -lgip
```

3. Create a configuration file in /home/apps/nvidia-mig-parted/config.yaml. Get a sample from [here](#).
4. Apply the desired profiles. For example, to activate a profile named all-balanced, use below command:

```
sudo mig-parted -f /home/apps/nvidia-mig-parted/config.yaml -c all-balanced
```

5. Verify that the MIG profile has been applied:

```
sudo nvidia-smi mig -lgi
```

Automating MIG profiling on reboot

By default, mig-parted would not survive reboot. To reapply MIG parted on each reboot:

1. Create a systemd service:

```
sudo systemctl edit --full --force nvidia-mig-parted.service
```

2. Fill up the file with content from [nvidia-mig-parted.service](#).

3. Save and exit.

4. Reload systemd daemon:

```
sudo systemctl daemon-reload
```

5. Start the service, and enable it to start on boot:

```
sudo systemctl enable --now nvidia-mig-parted
```

6. Verify that the service is now started, and the MIG profile has been activated:

```
sudo systemctl status nvidia-mig-parted; sudo nvidia-smi mig -lgi
```

Course Wrap-up & Enterprise Next Steps

Core Competencies Mastered

- **Day 1: Foundations & Storage**
 - Terminal navigation, pipes, stream redirection, nano.
 - Multi-user security, sudo, and POSIX permissions (`chmod 755/644/600`).
 - Dual-ecosystem package management (`apt` vs `dnf`).
 - Storage telemetry (`df` / `du`), loopback disks, ext4 vs XFS.
- **Day 2: Operations, Security & HPC**
 - Process lifecycle, signals (`SIGTERM` / `SIGKILL`), and systemd units.
 - Network sockets, port binding, and Ed25519 SSH keys.
 - Logging observability with `systemd-journald` and `grep` .
 - Datacenter NVIDIA GPU monitoring and MIG hardware partitioning.

? Questions & Discussion

Practical Linux System Administration Training

Participant Lab Guide: `LINUX_TRAINEE_WORKBOOK.md`

Course Syllabus & Timetable: `timetable.md` / `README.md`

Slide Deck: `SLIDES.md`

Repository materials dedicated under CC0 1.0 Universal Public Domain.